

# **IA Aplicada com LLMs**

**Aula 04 — O que é um agente, e onde eles estão**

## Onde paramos

Vocês sabem **construir** um laço de agente.

Declarar ferramentas, ler a chamada, executar, devolver o resultado.

O que vocês não têm é **repertório**.

## Hoje, quatro perguntas

1. **O que é um agente** — e o que ele não é
2. **Que tipos existem** — e que nomes de arquitetura vocês vão encontrar
3. **Para que serve** — os eixos de ganho, e como se medem
4. **Onde eles estão de verdade** — e quais funcionam

# **Parte 1**

**Antes de tudo: três palavras**

**Vocês vão ler material de mercado hoje**

E nele estas três palavras aparecem trocadas entre si:

**bot · assistente de IA · agente de IA**

**E a diferença é o que decide se um caso nos interessa**

|                     | <b>Bot</b>        | <b>Assistente</b> | <b>Agente</b>            |
|---------------------|-------------------|-------------------|--------------------------|
| <b>Autonomia</b>    | mínima            | média             | <b>máxima</b>            |
| <b>Complexidade</b> | interação simples | tarefa simples    | <b>fluxo complexo</b>    |
| <b>Aprendizado</b>  | pouco ou nenhum   | algum             | adapta com o tempo       |
| <b>Iniciativa</b>   | reage             | reage             | pode ser <b>proativo</b> |

## O eixo que faz o trabalho é o primeiro

**Bot** — segue regras pré-programadas

**Assistente** — depende de **comando e direção** do usuário

**Agente** — opera e decide **por conta própria** para atingir um objetivo

É a mesma régua do espectro de autonomia da Aula 01.

## **E um eixo novo: iniciativa**

**Reativo** — só age quando alguém pede.

**Proativo** — vive em segundo plano, acionado por **evento**.

Isso muda o desenho: alguém precisa decidir **quando ele acorda** —  
e o custo existe mesmo sem ninguém olhando.



## As características que a indústria atribui a um agente

| Característica                     | Onde vocês viram          |
|------------------------------------|---------------------------|
| Raciocínio e planejamento          | Aula 03 — CoT e o laço    |
| Memória (inclusive de longo prazo) | Aula 03 · aula de memória |
| Uso de ferramentas                 | Aula 03 — tool calling    |
| Autonomia                          | Aula 01 · próxima aula    |
| Multimodalidade                    | trabalho, se o caso pedir |
| Coordenação entre agentes          | Parte 3 do trabalho       |

## Duas leituras dessa tabela

**A primeira:** é um bom mapa do curso.

Cinco das seis são assunto de uma aula. Vocês já viram três.

**A segunda, mais útil:** é uma lista de **capacidades**,  
não de requisitos.

Nenhum sistema real tem as seis — e **não precisa** ter  
para ser um agente.

## Por que isso abre a aula

Quando um fornecedor chama de "agente" algo que segue regras pré-programadas...

**...ele está usando a palavra do jeito que a tabela proíbe**

Isso tem nome — *agent washing* — e voltaremos a ele.

## **E uma ressalva, para sermos coerentes**

A página que acabamos de usar é de um fornecedor de nuvem.

**Explicador de tema serve para vocabulário.**

**Não serve para evidência.**

Definir e classificar é o que este gênero faz bem.

Provar que alguém implantou é o que ele não faz.

Vale para tudo que vocês vão ler hoje: **quem publicou, e o que exatamente foi medido?**

## **Parte 2**

**O mapa: tipos e onde eles estão**

## A taxonomia que vocês vão encontrar

|          |        |                                          |
|----------|--------|------------------------------------------|
| CUSTOMER | agents | – falam com o cliente final              |
| EMPLOYEE | agents | – apoiam o funcionário por dentro        |
| CODE     | agents | – escrevem, revisam e corrigem software  |
| DATA     | agents | – consultam, cruzam e resumem dados      |
| CREATIVE | agents | – produzem conteúdo e texto de marketing |
| SECURITY | agents | – detectam, triam e remediam incidentes  |

11 setores × 6 tipos · mais de mil casos catalogados

## Mas repare

É uma taxonomia por **destinatário**, não por **arquitetura**.

Diz *para quem* o agente trabalha. Não diz *como* ele é feito.

O material de mercado descreve o destinatário e **omite a arquitetura**.

Ler a arquitetura é trabalho de vocês — e vocês já têm o vocabulário.

## Onde a adoção realmente está

**Por função:** TI, gestão de conhecimento, engenharia de software.

**Por setor:** bancos e seguros à frente; saúde e governo atrás.

**Por porte:** empresas de tecnologia muito à frente;  
organizações pequenas praticamente paradas.



## **E a governança não acompanhou**

Cerca de **um terço** das organizações relata maturidade razoável em estratégia e governança de IA agêntica.

Guardem esta frase.

Ela volta com número brasileiro, e pior.

## **Parte 3**

**Para que serve**

**"Agentes são o futuro" não é um benefício**

E a pergunta que qualquer chefe vai fazer tem resposta concreta:

**"o que isso me dá?"**

## Os três eixos do fornecedor

**Produtividade** — a tarefa repetitiva sai do caminho.  
O mais citado e o mais fácil de medir.

**Personalização** — o cliente é entendido, não roteirizado.  
Aparece como satisfação e conversão, não como hora poupada.

**Decisão** — o agente decide em vez de sugerir, onde a  
decisão autônoma cria valor imediato.

## Google Cloud · 2025 ROI of AI Report

- **74%** dos executivos relatam ROI no primeiro ano
- entre os que relatam ganho de produtividade,  
**39% viram a produtividade pelo menos dobrar**

*(Fornecedor sobre o próprio mercado, sem metodologia publicada.  
Ordem de grandeza, não promessa.)*

## **E dois eixos que vendem melhor**

**Velocidade de processo** — não é fazer mais no mesmo tempo, é o mesmo trabalho terminando antes.

*32% mais rápido na edição de conteúdo · 46% na criação · tempo de resposta a ameaça caindo pela metade*

**Redução de erro e retrabalho** — o menos anunciado, e o mais valioso onde o erro custa muita e não hora.

## O benefício que ninguém anuncia

Para construir o agente, vocês são **obrigados a escrever a regra**.

Qual é o teto. O que é atraso. Quando escala para humano.

E quase sempre se descobre que a regra **nunca esteve escrita** —  
vivia na cabeça de três pessoas, cada uma com uma versão.

## Consequência prática

# Parte do benefício vem sem LLM nenhum

A rota de regra do roteador — o `if` que resolve o caso claro — responde pela maioria do volume.

E ela só existiu porque alguém precisou escrever a regra.



## Como cada eixo se mede

| Eixo             | Unidade                                     |
|------------------|---------------------------------------------|
| Produtividade    | itens por pessoa por dia · horas devolvidas |
| Cobertura        | % resolvido sem humano                      |
| Redução de carga | fila que não chegou à pessoa                |
| Velocidade       | tempo do início ao fim de um caso           |
| Eficiência       | custo ou esforço por transação              |
| Receita          | conversão, ticket, posição de busca         |
| <b>Erro</b>      | taxa antes × depois · horas de correção     |

A última linha é a mais interessante

**Quase nenhum caso publica redução de erro**

E não é porque não acontece.

É porque medir exige ter medido **antes** — e quase ninguém mediu.

Se o seu tema permitir esse número, vocês têm  
o argumento mais forte da mesa.

## O gabarito de cada caso

1. o que a empresa **fez**
2. o **número** que ela divulgou
3. que **padrão de arquitetura** provavelmente está por trás
4. **por que funcionou**

O item 3 é inferência — nenhuma empresa publica o diagrama.

Fazer a inferência é o exercício.

**E um padrão que se repete até o fim**

## **Quase nenhum "agente" de sucesso é um agente**

A maioria é **roteador + workflow**, com a autonomia confinada a um caminho estreito.

É a régua da Aula 01 acontecendo no mundo:  
**use a menor autonomia que resolve.**

# **Parte 4**

## **Code agents**

## O número mais sólido da aula

JetBrains · Developer Ecosystem Survey 2026 · >**15.000** devs

**90% semanalmente**

**68% diariamente**

Nenhum outro tipo chega perto.

**A pergunta interessante não é *quanto***

**Por que aqui primeiro?**

## Porque o ambiente contradiz o modelo

O teste passa ou falha. O compilador aceita ou recusa.

O programa roda ou estoura.

É a descoberta mais importante desta aula:

o ambiente devolve **feedback verificável**

Um agente só se corrige onde existe algo capaz de dizer que ele errou.



## Compare

**"Corrigir um bug"** → o teste diz. Feedback em segundos, de graça.

**"Escrever um texto persuasivo"** → nada diz se convenceu.

O laço não tem do que se corrigir.

A lição para o case de vocês

## Procurem o verificador antes de procurar o problema

Se o domínio tem um "teste que passa ou falha", vocês escolheram um domínio onde agentes funcionam **hoje**.

Se não tem, vocês vão passar o semestre sem saber se está certo.  
E eu também não vou saber.

# **Parte 5**

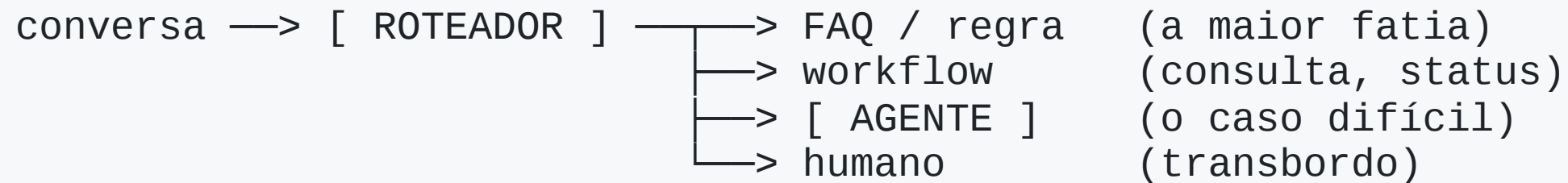
## **Os outros cinco tipos**

## Customer agents

| Empresa     | Número                                         |
|-------------|------------------------------------------------|
| Commerzbank | ~2 milhões de conversas, <b>70%</b> sem humano |
| NoBroker    | agentes cobrem <b>25–40%</b> das chamadas      |
| LUXGEN      | <b>–30%</b> de carga nos atendentes            |

Nenhum é 100%. Todos descrevem **uma fatia**.

## Customer agents — a arquitetura



É o desenho que vocês vão construir no laboratório da próxima aula.

E a rota mais valiosa é a que **não chama o modelo**.

## E cuidado com a definição

"Resolvida sem humano" é régua da própria empresa.

Costuma significar *terminou sem transbordo* —  
o que inclui **o cliente que desistiu**.

Quando o número depende de uma definição, procure a definição.

## Employee agents

**IBM AskHR** — ~90 automações de RH, **11,5 mi de interações** em 2024, **94% de contenção**, **-75% de tickets** desde 2016

**United Wholesale Mortgage** — produtividade de analistas de crédito **mais que dobrada** em 9 meses

**Uber** — sumarização de comunicações com usuários

**Intuit / TurboTax** — autofill dos 10 formulários fiscais mais comuns

## Mas "94% de contenção"...

...é a IBM falando da IBM.

E **contenção** é a régua da própria empresa:  
*a conversa não escalou para um humano* — o que inclui quem **desistiu** e foi resolver por outro canal.

Útil. E não é o que parece.



**O que não aparece na manchete**

**O humano continua decidindo**

O analista não foi substituído — recebe o caso pré-analisado.

O erro custa segundos de conferência, não prejuízo.

É a arquitetura mais tolerante a falha que existe.

Se o case de vocês couber aqui, é mais fácil de terminar.

## Data agents

| Empresa | Número                                         |
|---------|------------------------------------------------|
| Geotab  | 4,7 milhões de veículos, bilhões de pontos/dia |
| Moglix  | 4× em eficiência de <i>sourcing</i>            |
| Domina  | +80% de acesso a dados, +15% de efetividade    |

Padrão: **orquestrador-trabalhador** — e o que mais precisa de **teto**.

## Ressalva de leitura

"4× de eficiência" e "+80% de acesso" são **autodeclarados, sem linha de base publicada.**

4× em relação a quê? Medido como?

Servem para saber que o tipo funciona.

Não servem para prometer o mesmo ganho a ninguém.

## **Creative × Security — os dois extremos**

**Creative** — AdVon: 93.673 produtos em menos de um mês.

Errar é barato; revisão humana é o padrão.

**Security** — falso positivo custa o analista, falso negativo custa o incidente. Menor tolerância a erro dos seis tipos.

**A variável que os separa não é a tecnologia**

**É o custo do erro**

Quanto mais caro o erro, menor a autonomia aceitável — independentemente do que o modelo é capaz de fazer.

É a régua do espectro de autonomia da Aula 01, agora com a consequência do erro entrando na conta.



## **Parte 6**

**A categoria que a taxonomia não tem**

## Repare nos seis tipos

Cliente · funcionário · código · dados · criativo · segurança

Todos classificam **por destinatário**.

E todos os seis destinatários são **corporativos**.



**Falta aquele cujo usuário é**  
**você**

## Assistente pessoal local

**OpenClaw** — runtime auto-hospedado que funciona como **gateway de mensagens**: WhatsApp, Telegram, Discord, Slack, Signal, iMessage, Matrix, Teams → um agente que lê os seus arquivos e **roda comandos na sua máquina**.

**Hermes Agent** (Nous Research, aberto, fev/2026) — conecta-se a um endpoint de modelo, tem memória persistente e **cria as próprias skills**. Roda offline com Ollama; do laptop ao serverless.

**Três coisas que só esta categoria mostra**

# 1. Proatividade de verdade

Todos os casos corporativos desta aula são **reativos**: alguém abre um chamado, manda mensagem, submete um lote.

O **briefing matinal** do OpenClaw é o único exemplo aqui de um agente que **começa a conversa**.

E isso muda o desenho: alguém decide **quando ele acorda**, e o custo existe sem ninguém olhando.

## 2. Memória de longo prazo, em uso

É a característica do §1 que **nenhum** caso corporativo ilustrou.

Aqui é o argumento de venda central:

*quanto mais tempo roda, melhor te conhece*

### **3. Uso pessoal que é uso profissional**

Acompanhar repositório · disparar deploy · revisar código · ver log

Tudo pelo Telegram, em vez de trocar de aba.

*"Algum teste falhou no último commit?"*

**Code agent + data agent na mesma caixa, com um usuário só.**

## **E o risco é de outra ordem**

Um agente com acesso ao seu shell, aos seus arquivos e às suas credenciais, alcançável por um canal de mensagem.

Não é hipótese. Em 2026:

## Os números da exposição

- **+30 mil instâncias** do OpenClaw expostas na internet aberta (Bitsight; varreduras posteriores acharam mais)
- a causa principal: escutar em **todas as interfaces** em vez de só no `localhost`
- **ClawJacked** — site malicioso sequestrava o agente local via WebSocket e emitia comandos com privilégio de admin
- **milhares** de instalações com senha e chave de API expostas (Illumio)



A lição é a mesma da próxima parte

## O problema não é o modelo

É o **acesso** que se deu a ele — e a **configuração** com que se deu.

Um agente na sua máquina, com o seu shell,  
é a **versão doméstica do caso Replit**.

## Para o tema de vocês

**Instalem um esta semana.** É o jeito mais rápido de sentir o que um agente com ferramentas de verdade faz — inclusive o susto.

**Mas como tema do trabalho, cuidado:** *"montar meu assistente pessoal"* cai no anti-padrão do **produto de terceiro** — o esforço vira integração, e não arquitetura de agente.

Se for por aqui: **uma** capacidade, com verificador.

## **Parte 7**

**O que eles têm em comum**

## As seis características

| Característica                        | Onde aparece                        |
|---------------------------------------|-------------------------------------|
| Tarefa repetitiva, <b>volume alto</b> | 2 milhões de conversas              |
| <b>Feedback verificável</b>           | o teste que passa ou falha          |
| Tolerância a erro pequeno             | texto de produto sim; contenção não |
| <b>Ação reversível</b> ou confirmação | o analista que confere              |
| Dado acessível por API                | todos, sem exceção                  |
| <b>Um número</b> que define sucesso   | 70%, 4x, -30%                       |

## **E o que NÃO está na lista**

Qual modelo foi usado.

Qual framework.

Quantos parâmetros.

**Nenhum case study atribui o resultado ao modelo.**

Todos atribuem à integração — que é o que o MIT apontou como causa dos 95%.

## A frase que resume o deck

O mais difícil hoje não é a inteligência do modelo.

É o **acesso seguro e confiável aos sistemas de produção.**

Isso deveria mudar a estimativa de esforço do projeto de vocês.

A parte que parece o projeto — prompt, laço, modelo — é a menor.

# Síntese

- **Bot × assistente × agente** se separam por **autonomia**
- A lista de características de um agente é de **capacidades**, não de requisitos
- A taxonomia do mercado é por **destinatário** — a arquitetura vocês inferem
- **Code agents** lidera porque o ambiente devolve **feedback verificável**
- **Customer agents**: volume alto, autonomia baixa — roteador + workflow
- **Employee agents**: o humano é o verificador com autoridade
- **Data agents**: orquestrador-trabalhador, e precisa de teto
- **Creative × Security**: a variável é o **custo do erro**
- Seis características comuns — e o **verificador** é a que decide
- Nenhum caso atribui o resultado ao modelo. Todos, à **integração**

## E amanhã os nomes viram código

Vocês vão sair daqui sabendo **ler** uma arquitetura de agente.

A próxima aula ensina a **construir** cada uma delas —  
com orçamento, salvaguarda e o que fazer quando falha.